Namespace PolylineAlgorithm

Classes

PolylineDecoder

Performs decoding of encoded polyline strings into a sequence of geographic coordinates.

PolylineEncoder

Provides methods to encode a set of geographic coordinates into a polyline string. This class implements the IPolylineEncoder interface.

Structs

Coordinate

Represents a geographic coordinate as a pair of latitude and longitude values.

Polyline

Represents an immutable, read-only encoded polyline string. Provides methods for creation, inspection, and conversion of polyline data from various character sources.

Struct Coordinate

Namespace: PolylineAlgorithm

Assembly: PolylineAlgorithm.dll

Represents a geographic coordinate as a pair of latitude and longitude values.

```
public readonly struct Coordinate : IEquatable<Coordinate>
```

Implements
IEquatable<Coordinate>

Inherited Members
object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

Coordinate()

Initializes a new instance of the Coordinate struct with default values (0) for Latitude and Longitude.

```
public Coordinate()
```

Coordinate(double, double)

Initializes a new instance of the Coordinate struct with the specified latitude and longitude values.

```
public Coordinate(double latitude, double longitude)
```

Parameters

`latitude` double

The latitude component, in degrees. Must be between -90 and 90, inclusive.

`longitude` double

The longitude component, in degrees. Must be between -180 and 180, inclusive.

Exceptions

ArgumentOutOfRangeException

Thrown when `latitude` is less than -90 or greater than 90, or when `longitude` is less than -180 or greater than 180.

Properties

Latitude

Gets the latitude component of the coordinate, in degrees.

```
public double Latitude { get; }
```

Property Value

double

Longitude

Gets the longitude component of the coordinate, in degrees.

```
public double Longitude { get; }
```

Property Value

double

Methods

Equals(Coordinate)

Indicates whether this coordinate is equal to another Coordinate instance.

```
public bool Equals(Coordinate other)
```

Parameters

`other` Coordinate

The coordinate to compare with this instance.

Returns

bool

true if both latitude and longitude are equal; otherwise, false.

Equals(object?)

Indicates whether this instance and a specified object are equal.

```
public override bool Equals(object? obj)
```

Parameters

`obj` object

The object to compare with the current instance.

Returns

bool

true if `obj` and this instance are the same type and represent the same value; otherwise, false.

GetHashCode()

Returns the hash code for this instance.

```
public override int GetHashCode()
```

Returns

int

A 32-bit signed integer that is the hash code for this instance.

IsDefault()

Determines whether this coordinate is the default value (both Latitude and Longitude are 0).

```
public bool IsDefault()
```

Returns

bool

true if both latitude and longitude are 0; otherwise, false.

ToString()

Returns a string representation of this coordinate in the format: `{ Latitude: [double], Longitude: [double] }`.

```
public override string ToString()
```

Returns

string

A string representation of the coordinate.

Operators

operator ==(Coordinate, Coordinate)

Determines whether two Coordinate instances are equal.

```
public static bool operator ==(Coordinate left, Coordinate right)
```

Parameters

`left` Coordinate

The first coordinate to compare.

`right` Coordinate

The second coordinate to compare.

Returns

bool

true if both coordinates are equal; otherwise, false.

operator !=(Coordinate, Coordinate)

Determines whether two Coordinate instances are not equal.

```
public static bool operator !=(Coordinate left, Coordinate right)
```

Parameters

`left` Coordinate

The first coordinate to compare.

`right` Coordinate

The second coordinate to compare.

Returns

bool

true if the coordinates are not equal; otherwise, false.

Struct Polyline

Namespace: PolylineAlgorithm

Assembly: PolylineAlgorithm.dll

Represents an immutable, read-only encoded polyline string. Provides methods for creation, inspection, and conversion of polyline data from various character sources.

```
public readonly struct Polyline : IEquatable<Polyline>
```

Implements
IEquatable<Polyline>

Inherited Members
object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

Polyline()

Initializes a new, empty instance of the Polyline struct.

```
public Polyline()
```

Properties

IsEmpty

Gets a value indicating whether this Polyline is empty.

```
public bool IsEmpty { get; }
```

Property Value

bool

Length

Gets the length of the polyline in characters.

```
public long Length { get; }
```

Property Value

long

Methods

CopyTo(char[])

Copies the characters of this polyline to the specified destination array.

```
public void CopyTo(char[] destination)
```

Parameters

`destination` char[]

The destination array to copy the characters to.

Exceptions

ArgumentNullException

Thrown if `destination` is `null`.

ArgumentException

Thrown if the length of `destination` does not match the polyline's length.

Equals(Polyline)

Determines whether the current polyline is equal to another polyline.

```
public bool Equals(Polyline other)
```

Parameters

`other` Polyline

The polyline to compare with the current polyline.

Returns

bool

true if the polylines are equal; otherwise, false.

Equals(object)

Indicates whether this instance and a specified object are equal.

```
public override bool Equals(object obj)
```

Parameters

`obj` object

The object to compare with the current instance.

Returns

bool

true if `obj` and this instance are the same type and represent the same value; otherwise, false.

FromCharArray(char[])

Creates a Polyline from a Unicode character array.

```
public static Polyline FromCharArray(char[] polyline)
```

Parameters

`polyline` char[]

A Unicode character array representing an encoded polyline.

Returns

Polyline

The Polyline instance corresponding to the specified character array.

Exceptions

ArgumentNullException

Thrown if `polyline` is `null`.

FromMemory(ReadOnlyMemory<char>)

Creates a Polyline from a read-only memory region of characters.

```
public static Polyline FromMemory(ReadOnlyMemory<char> polyline)
```

Parameters

`polyline` ReadOnlyMemory<char>

A read-only memory region representing an encoded polyline.

Returns

Polyline

The Polyline instance corresponding to the specified memory region.

FromString(string)

Creates a Polyline from a string.

```
public static Polyline FromString(string polyline)
```

Parameters

`polyline` string

A string representing an encoded polyline.

Returns

Polyline

The Polyline instance corresponding to the specified string.

Exceptions

ArgumentNullException

Thrown if `polyline` is `null`.

GetHashCode()

Returns the hash code for this instance.

```
public override int GetHashCode()
```

Returns

int

A 32-bit signed integer that is the hash code for this instance.

ToString()

Returns a string representation of the polyline.

```
public override string ToString()
```

Returns

string

A string containing the characters of the polyline, or an empty string if the polyline is empty.

Operators

operator ==(Polyline, Polyline)

Determines whether two Polyline instances are equal.

```
public static bool operator ==(Polyline left, Polyline right)
```

Parameters

`left` Polyline

The first polyline to compare.

`right` Polyline

The second polyline to compare.

Returns

bool

true if the polylines are equal; otherwise, false.

operator !=(Polyline, Polyline)

Determines whether two Polyline instances are not equal.

```
public static bool operator !=(Polyline left, Polyline right)
```

Parameters

`left` Polyline

The first polyline to compare.

`right` Polyline

The second polyline to compare.

Returns

bool

true if the polylines are not equal; otherwise, false.

Class PolylineDecoder

Namespace: PolylineAlgorithm

Assembly: PolylineAlgorithm.dll

Performs decoding of encoded polyline strings into a sequence of geographic coordinates.

```
public sealed class PolylineDecoder : AbstractPolylineDecoder<Polyline, Coordinate>,
IPolylineDecoder<Polyline, Coordinate>
```

Inheritance

object ← AbstractPolylineDecoder<Polyline, Coordinate> ← PolylineDecoder

Implements

IPolylineDecoder<Polyline, Coordinate>

Inherited Members

AbstractPolylineDecoder<Polyline, Coordinate>.Options , AbstractPolylineDecoder<Polyline, Coordinate>.Decode(Polyline) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Extension Methods

PolylineDecoderExtensions.Decode(IPolylineDecoder<Polyline, Coordinate>, char[]) , PolylineDecoderExtensions.Decode(IPolylineDecoder<Polyline, Coordinate>, ReadOnlyMemory<char>) , PolylineDecoderExtensions.Decode(IPolylineDecoder<Polyline, Coordinate>, string)

Methods

CreateCoordinate(double, double)

Creates a coordinate instance from the given latitude and longitude values.

```
protected override Coordinate CreateCoordinate(double latitude, double longitude)
```

Parameters

`latitude` double

The latitude value.

`longitude` double

The longitude value.

Returns

Coordinate

A coordinate instance of type Coordinate.

GetReadOnlyMemory(Polyline)

Converts the provided polyline instance into a ReadOnlyMemory<T> for decoding.

```
protected override ReadOnlyMemory<char> GetReadOnlyMemory(Polyline polyline)
```

Parameters

`polyline` Polyline

Returns

ReadOnlyMemory<char>

Class PolylineEncoder

Namespace: PolylineAlgorithm

Assembly: PolylineAlgorithm.dll

Provides methods to encode a set of geographic coordinates into a polyline string. This class implements the IPolylineEncoder interface.

```
public sealed class PolylineEncoder : AbstractPolylineEncoder<Coordinate, Polyline>,
IPolylineEncoder<Coordinate, Polyline>
```

Inheritance

object ← AbstractPolylineEncoder<Coordinate, Polyline> ← PolylineEncoder

Implements

IPolylineEncoder<Coordinate, Polyline>

Inherited Members

AbstractPolylineEncoder<Coordinate, Polyline>.Options , AbstractPolylineEncoder<Coordinate, Polyline>.Encode(IEnumerable<Coordinate>) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Extension Methods

PolylineEncoderExtensions.Encode(IPolylineEncoder<Coordinate, Polyline>, Coordinate[]) , PolylineEncoderExtensions.Encode(IPolylineEncoder<Coordinate, Polyline>, ICollection<Coordinate>)

Methods

CreatePolyline(ReadOnlyMemory<char>)

Creates a polyline instance from the provided read-only sequence of characters.

```
protected override Polyline CreatePolyline(ReadOnlyMemory<char> polyline)
```

Parameters

`polyline` ReadOnlyMemory<char>

A ReadOnlySequence<T> containing the encoded polyline characters.

Returns

Polyline

An instance of Polyline representing the encoded polyline.

GetLatitude(Coordinate)

Extracts the latitude value from the specified coordinate.

```
protected override double GetLatitude(Coordinate coordinate)
```

Parameters

`coordinate` Coordinate

Returns

double

The latitude value as a double.

GetLongitude(Coordinate)

Extracts the longitude value from the specified coordinate.

```
protected override double GetLongitude(Coordinate coordinate)
```

Parameters

`coordinate` Coordinate

Returns

double

The longitude value as a double.

Namespace PolylineAlgorithm.Abstraction

Classes

AbstractPolylineDecoder<TPolyline, TCoordinate>

Decodes encoded polyline strings into sequences of geographic coordinates. Implements the IPolylineDecoder<TPolyline, TCoordinate> interface.

AbstractPolylineEncoder<TCoordinate, TPolyline>

Provides functionality to encode a collection of geographic coordinates into an encoded polyline string. Implements the IPolylineEncoder<TCoordinate, TPolyline> interface.

InvalidPolylineException

Exception thrown when a polyline is determined to be malformed or invalid during processing.

PolylineEncoding

Provides methods for encoding and decoding polyline data, as well as utilities for normalizing and denormalizing geographic coordinate values.

PolylineEncodingOptions

Options for configuring polyline encoding.

PolylineEncodingOptionsBuilder

Interfaces

IPolylineDecoder<TPolyline, TCoordinate>

Defines a contract for decoding an encoded polyline into a collection of geographic coordinates.

IPolylineEncoder<TCoordinate, TPolyline>

Defines a contract for encoding a collection of geographic coordinates into an encoded polyline string.

IPolylineEncodingOptionsBuilder

Enums

PolylineEncoding.ValueType

Represents the type of a geographic coordinate value.

Class AbstractPolylineDecoder<TPolyline, TCoordinate>

Namespace: PolylineAlgorithm.Abstraction

Assembly: PolylineAlgorithm.Abstraction.dll

Decodes encoded polyline strings into sequences of geographic coordinates. Implements the IPolylineDecoder<TPolyline, TCoordinate> interface.

```
public abstract class AbstractPolylineDecoder<TPolyline, TCoordinate> :
IPolylineDecoder<TPolyline, TCoordinate>
```

Type Parameters

`TPolyline`

`TCoordinate`

Inheritance

object ← AbstractPolylineDecoder<TPolyline, TCoordinate>

Implements

IPolylineDecoder<TPolyline, TCoordinate>

Derived

PolylineDecoder

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

AbstractPolylineDecoder()

```
public AbstractPolylineDecoder()
```

AbstractPolylineDecoder(PolylineEncodingOptions)

```
public AbstractPolylineDecoder(PolylineEncodingOptions options)
```

Parameters

`options` PolylineEncodingOptions

Properties

Options

Gets the encoding options used by this polyline encoder.

```
public PolylineEncodingOptions Options { get; }
```

Property Value

PolylineEncodingOptions

Methods

CreateCoordinate(double, double)

Creates a coordinate instance from the given latitude and longitude values.

```
protected abstract TCoordinate CreateCoordinate(double latitude, double longitude)
```

Parameters

`latitude` double

The latitude value.

`longitude` double

The longitude value.

Returns

TCoordinate

A coordinate instance of type `TCoordinate`.

Decode(TPolyline)

Decodes an encoded `TPolyline` into a sequence of `TCoordinate` instances.

```
public IEnumerable<TCoordinate> Decode(TPolyline polyline)
```

Parameters

`polyline` TPolyline

The `TPolyline` instance containing the encoded polyline string to decode.

Returns

IEnumerable<TCoordinate>

An IEnumerable<T> of `TCoordinate` representing the decoded latitude and longitude pairs.

Exceptions

ArgumentException

Thrown when `polyline` is empty.

InvalidPolylineException

Thrown when the polyline format is invalid or malformed at a specific position.

GetReadOnlyMemory(TPolyline?)

Converts the provided polyline instance into a ReadOnlySequence<T> for decoding.

```
protected abstract ReadOnlyMemory<char> GetReadOnlyMemory(TPolyline? polyline)
```

Parameters

`polyline` TPolyline

The polyline instance to convert. May be `null`.

Returns

ReadOnlyMemory<char>

A ReadOnlySequence<T> representing the encoded polyline data.

Class AbstractPolylineEncoder<TCoordinate, TPolyline>

Namespace: PolylineAlgorithm.Abstraction

Assembly: PolylineAlgorithm.Abstraction.dll

Provides functionality to encode a collection of geographic coordinates into an encoded polyline string. Implements the IPolylineEncoder<TCoordinate, TPolyline> interface.

```
public abstract class AbstractPolylineEncoder<TCoordinate, TPolyline> :
IPolylineEncoder<TCoordinate, TPolyline>
```

Type Parameters

`TCoordinate`

`TPolyline`

Inheritance

object ← AbstractPolylineEncoder<TCoordinate, TPolyline>

Implements

IPolylineEncoder<TCoordinate, TPolyline>

Derived

PolylineEncoder

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

AbstractPolylineEncoder()

```
public AbstractPolylineEncoder()
```

AbstractPolylineEncoder(PolylineEncodingOptions)

```
public AbstractPolylineEncoder(PolylineEncodingOptions options)
```

Parameters

`options` PolylineEncodingOptions

Properties

Options

Gets the encoding options used by this polyline encoder.

```
public PolylineEncodingOptions Options { get; }
```

Property Value

PolylineEncodingOptions

Methods

CreatePolyline(ReadOnlyMemory<char>)

Creates a polyline instance from the provided read-only sequence of characters.

```
protected abstract TPolyline CreatePolyline(ReadOnlyMemory<char> polyline)
```

Parameters

`polyline` ReadOnlyMemory<char>

A ReadOnlySequence<T> containing the encoded polyline characters.

Returns

TPolyline

An instance of `TPolyline` representing the encoded polyline.

Encode(IEnumerable<TCoordinate>)

Encodes a collection of `TCoordinate` instances into an encoded `TPolyline` string.

```
public TPolyline Encode(IEnumerable<TCoordinate> coordinates)
```

Parameters

`coordinates` IEnumerable<TCoordinate>

The collection of `TCoordinate` objects to encode.

Returns

TPolyline

An instance of `TPolyline` representing the encoded coordinates. Returns default if the input collection is empty or null.

Exceptions

ArgumentNullException

Thrown when `coordinates` is null.

ArgumentException

Thrown when `coordinates` is an empty enumeration.

GetLatitude(TCoordinate?)

Extracts the latitude value from the specified coordinate.

```
protected abstract double GetLatitude(TCoordinate? current)
```

Parameters

`current` TCoordinate

The coordinate from which to extract the latitude.

Returns

double

The latitude value as a double.

GetLongitude(TCoordinate?)

Extracts the longitude value from the specified coordinate.

```
protected abstract double GetLongitude(TCoordinate? current)
```

Parameters

`current` TCoordinate

The coordinate from which to extract the longitude.

Returns

double

The longitude value as a double.

Interface IPolylineDecoder<TPolyline, TCoordinate>

Namespace: PolylineAlgorithm.Abstraction

Assembly: PolylineAlgorithm.Abstraction.dll

Defines a contract for decoding an encoded polyline into a collection of geographic coordinates.

```
public interface IPolylineDecoder<TPolyline, TCoordinate>
```

Type Parameters

`TPolyline`

`TCoordinate`

Methods

Decode(TPolyline)

Decodes the specified encoded polyline into a sequence of geographic coordinates.

```
IEnumerable<TCoordinate> Decode(TPolyline polyline)
```

Parameters

`polyline` TPolyline

The `TPolyline` instance containing the encoded polyline string to decode.

Returns

IEnumerable<TCoordinate>

An IEnumerable<T> of `TCoordinate` representing the decoded latitude and longitude pairs.

Interface IPolylineEncoder<TCoordinate, TPolyline>

Namespace: PolylineAlgorithm.Abstraction

Assembly: PolylineAlgorithm.Abstraction.dll

Defines a contract for encoding a collection of geographic coordinates into an encoded polyline string.

```
public interface IPolylineEncoder<TCoordinate, TPolyline>
```

Type Parameters

`TCoordinate`

`TPolyline`

Methods

Encode(IEnumerable<TCoordinate>)

Encodes a sequence of geographic coordinates into an encoded polyline representation.

```
TPolyline Encode(IEnumerable<TCoordinate> coordinates)
```

Parameters

`coordinates` IEnumerable<TCoordinate>

The collection of `TCoordinate` instances to encode into a polyline.

Returns

TPolyline

A `TPolyline` containing the encoded polyline string that represents the input coordinates.

Interface IPolylineEncodingOptionsBuilder

Namespace: PolylineAlgorithm.Abstraction

Assembly: PolylineAlgorithm.Abstraction.dll

```
public interface IPolylineEncodingOptionsBuilder
```

Methods

Build()

```
PolylineEncodingOptions Build()
```

Returns

PolylineEncodingOptions

WithBufferSize(int)

```
IPolylineEncodingOptionsBuilder WithBufferSize(int maxBufferSize)
```

Parameters

`maxBufferSize` int

Returns

IPolylineEncodingOptionsBuilder

WithLoggerFactory(ILoggerFactory)

```
IPolylineEncodingOptionsBuilder WithLoggerFactory(ILoggerFactory loggerFactory)
```

Parameters

`loggerFactory` ILoggerFactory

Returns

IPolylineEncodingOptionsBuilder

Class InvalidPolylineException

Namespace: PolylineAlgorithm.Abstraction

Assembly: PolylineAlgorithm.Abstraction.dll

Exception thrown when a polyline is determined to be malformed or invalid during processing.

```
public sealed class InvalidPolylineException : Exception, ISerializable
```

Inheritance

object ← Exception ← InvalidPolylineException

Implements

ISerializable

Inherited Members

Exception.GetBaseException() , Exception.GetObjectData(SerializationInfo, StreamingContext) , Exception.GetType() , Exception.ToString() , Exception.Data , Exception.HelpLink , Exception.HResult , Exception.InnerException , Exception.Message , Exception.Source , Exception.StackTrace , Exception.TargetSite , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.ReferenceEquals(object, object)

Methods

Throw(long)

Throws an InvalidPolylineException with a message indicating the position in the polyline where the error was detected.

```
public static void Throw(long position)
```

Parameters

`position` long

The zero-based index in the polyline string where the error occurred.

Exceptions

InvalidPolylineException

Always thrown to indicate that the polyline is malformed at the specified position.

Class PolylineEncoding

Namespace: PolylineAlgorithm.Abstraction

Assembly: PolylineAlgorithm.Abstraction.dll

Provides methods for encoding and decoding polyline data, as well as utilities for normalizing and denormalizing geographic coordinate values.

```
public static class PolylineEncoding
```

Inheritance

object ← PolylineEncoding

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The PolylineEncoding class includes functionality for working with encoded polyline data, such as reading and writing encoded values, as well as methods for normalizing and denormalizing geographic coordinates. It also provides validation utilities to ensure values conform to expected ranges for latitude and longitude.

Methods

Denormalize(int, ValueType)

Converts a normalized integer value to its denormalized double representation based on the specified type.

```
public static double Denormalize(int value, PolylineEncoding.ValueType type)
```

Parameters

`value` int

The normalized integer value to be denormalized. Must be within the valid range for the specified `type`.

`type` PolylineEncoding.ValueType

The type that defines the valid range for `value`.

Returns

double

The denormalized double representation of the input value. Returns `0.0` if `value` is `0`.

Remarks

The denormalization process divides the input value by a predefined precision factor to produce the resulting double. Ensure that `value` is validated against the specified `type` before calling this method.

Exceptions

ArgumentOutOfRangeException

Thrown if `value` is outside the valid range for the specified `type`.

GetCharCount(int)

Determines the number of characters required to represent the specified integer value within predefined variance ranges.

```
public static int GetCharCount(int variance)
```

Parameters

`variance` int

The integer value for which the character count is calculated. Must be within the range of a 32-bit signed integer.

Returns

int

The number of characters required to represent the `variance` value, based on its magnitude. Returns a value between 1 and 6 inclusive.

Remarks

The method uses predefined ranges to efficiently determine the character count. Smaller values require fewer characters, while larger values require more. This method is optimized for performance using a switch expression.

Normalize(double, ValueType)

Normalizes a given numeric value based on the specified type and precision settings.

```
public static int Normalize(double value, PolylineEncoding.ValueType type)
```

Parameters

`value` double

The numeric value to normalize. Must be a finite number.

`type` PolylineEncoding.ValueType

The type against which the value is validated. Determines the acceptable range for the value.

Returns

int

An integer representing the normalized value. Returns `0` if the input value is `0.0`.

Remarks

This method validates the input value to ensure it is finite and within the acceptable range for the specified type. If the value is valid, it applies a normalization algorithm using a predefined precision factor.

Exceptions

ArgumentOutOfRangeException

Thrown if `value` is not a finite number or is outside the valid range for the specified `type`.

TryReadValue(ref int, ref ReadOnlyMemory<char>, ref int)

Attempts to read a value from the specified buffer and updates the variance.

```
public static bool TryReadValue(ref int variance, ref ReadOnlyMemory<char> buffer,
ref int position)
```

Parameters

`variance` int

A reference to the integer that will be updated based on the value read from the buffer.

`buffer` ReadOnlyMemory<char>

A reference to the read-only memory buffer containing the data to be processed.

`position` int

A reference to the current position within the buffer. The position is incremented as the method reads data.

Returns

bool

true if a value was successfully read and the end of the buffer was not reached; otherwise, false.

Remarks

This method processes the buffer starting at the specified position and attempts to decode a value. The decoded value is used to update the `variance` parameter. The method stops reading when a termination condition is met or the end of the buffer is reached.

TryWriteValue(int, ref Span<char>, ref int)

Attempts to write a value derived from the specified `variance` into the provided `buffer` at the given `position`.

```
public static bool TryWriteValue(int variance, ref Span<char> buffer, ref
int position)
```

Parameters

`variance` int

The integer value used to calculate the output to be written into the buffer.

`buffer` Span<char>

A reference to the span of characters where the value will be written.

`position` int

A reference to the current position in the buffer where writing begins. This value is updated to reflect the new position after writing.

Returns

bool

true if the value was successfully written to the buffer; otherwise, false.

Remarks

This method performs bounds checking to ensure that the buffer has sufficient space to accommodate the calculated value. If the buffer does not have enough space, the method returns false without modifying the buffer or position.

Enum PolylineEncoding.ValueType

Namespace: PolylineAlgorithm.Abstraction

Assembly: PolylineAlgorithm.Abstraction.dll

Represents the type of a geographic coordinate value.

```
public enum PolylineEncoding.ValueType
```

Fields

`Latitude = 0`

`Longitude = 1`

Remarks

This enumeration is used to specify whether a coordinate value represents latitude or longitude. Latitude values indicate the north-south position, while longitude values indicate the east-west position.

Class PolylineEncodingOptions

Namespace: PolylineAlgorithm.Abstraction

Assembly: PolylineAlgorithm.Abstraction.dll

Options for configuring polyline encoding.

```
public sealed class PolylineEncodingOptions
```

Inheritance

object ← PolylineEncodingOptions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

BufferSize

Gets the maximum buffer size for encoding operations.

```
public int BufferSize { get; }
```

Property Value

int

LoggerFactory

```
public ILoggerFactory LoggerFactory { get; }
```

Property Value

ILoggerFactory

MaxLength

```
public int MaxLength { get; }
```

Property Value

int

Methods

GetLoggerFor<T>()

```
public ILogger GetLoggerFor<T>()
```

Returns

ILogger

Type Parameters

T

Class PolylineEncodingOptionsBuilder

Namespace: PolylineAlgorithm.Abstraction

Assembly: PolylineAlgorithm.Abstraction.dll

```
public class PolylineEncodingOptionsBuilder : IPolylineEncodingOptionsBuilder
```

Inheritance

object ← PolylineEncodingOptionsBuilder

Implements

IPolylineEncodingOptionsBuilder

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Create()

Creates a new IPolylineEncodingOptionsBuilder instance for the specified coordinate type.

```
public static IPolylineEncodingOptionsBuilder Create()
```

Returns

IPolylineEncodingOptionsBuilder

An IPolylineEncodingOptionsBuilder instance for configuring polyline encoding options.

Namespace PolylineAlgorithm.Extensions

Classes

PolylineDecoderExtensions

Provides extension methods for the IPolylineDecoder<TPolyline, TCoordinate> interface to facilitate decoding encoded polylines.

PolylineEncoderExtensions

Provides extension methods for the IPolylineEncoder<TCoordinate, TPolyline> interface to facilitate encoding geographic coordinates into polylines.

Class PolylineDecoderExtensions

Namespace: PolylineAlgorithm.Extensions

Assembly: PolylineAlgorithm.dll

Provides extension methods for the IPolylineDecoder<TPolyline, TCoordinate> interface to facilitate decoding encoded polylines.

```
public static class PolylineDecoderExtensions
```

Inheritance

object ← PolylineDecoderExtensions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Decode(IPolylineDecoder<Polyline, Coordinate>, char[])

Decodes an encoded polyline represented as a character array into a sequence of geographic coordinates.

```
public static IEnumerable<Coordinate> Decode(this IPolylineDecoder<Polyline,
Coordinate> decoder, char[] polyline)
```

Parameters

`decoder` IPolylineDecoder<Polyline, Coordinate>

The IPolylineDecoder<TPolyline, TCoordinate> instance used to perform the decoding operation.

`polyline` char[]

The encoded polyline as a character array to decode.

Returns

IEnumerable<Coordinate>

An IEnumerable<T> containing the decoded latitude and longitude pairs.

Exceptions

ArgumentNullException

Thrown if `decoder` is null.

Decode(IPolylineDecoder<Polyline, Coordinate>, ReadOnlyMemory<char>)

```
public static IEnumerable<Coordinate> Decode(this IPolylineDecoder<Polyline,
Coordinate> decoder, ReadOnlyMemory<char> polyline)
```

Parameters

`decoder` IPolylineDecoder<Polyline, Coordinate>

`polyline` ReadOnlyMemory<char>

Returns

IEnumerable<Coordinate>

Decode(IPolylineDecoder<Polyline, Coordinate>, string)

Decodes an encoded polyline string into a sequence of geographic coordinates.

```
public static IEnumerable<Coordinate> Decode(this IPolylineDecoder<Polyline,
Coordinate> decoder, string polyline)
```

Parameters

`decoder` IPolylineDecoder<Polyline, Coordinate>

The IPolylineDecoder<TPolyline, TCoordinate> instance used to perform the decoding operation.

`polyline` string

The encoded polyline string to decode.

Returns

IEnumerable<Coordinate>

An IEnumerable<T> containing the decoded latitude and longitude pairs.

Exceptions

ArgumentNullException

Thrown if `decoder` is null.

Class PolylineEncoderExtensions

Namespace: PolylineAlgorithm.Extensions

Assembly: PolylineAlgorithm.dll

Provides extension methods for the IPolylineEncoder<TCoordinate, TPolyline> interface to facilitate encoding geographic coordinates into polylines.

```
public static class PolylineEncoderExtensions
```

Inheritance

object ← PolylineEncoderExtensions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Encode(IPolylineEncoder<Coordinate, Polyline>, Coordinate[])

Encodes an array of Coordinate instances into an encoded polyline.

```
public static Polyline Encode(this IPolylineEncoder<Coordinate, Polyline> encoder,
Coordinate[] coordinates)
```

Parameters

`encoder` IPolylineEncoder<Coordinate, Polyline>

The IPolylineEncoder instance used to perform the encoding operation.

`coordinates` Coordinate[]

The array of Coordinate objects to encode.

Returns

Polyline

A Polyline representing the encoded polyline string for the provided coordinates.

Exceptions

ArgumentNullException

Thrown if `encoder` is null.

Encode(IPolylineEncoder<Coordinate, Polyline>, ICollection<Coordinate>)

Encodes a collection of Coordinate instances into an encoded polyline.

```
public static Polyline Encode(this IPolylineEncoder<Coordinate, Polyline> encoder,
ICollection<Coordinate> coordinates)
```

Parameters

`encoder` IPolylineEncoder<Coordinate, Polyline>

The IPolylineEncoder<TCoordinate, TPolyline> instance used to perform the encoding operation.

`coordinates` ICollection<Coordinate>

The collection of Coordinate objects to encode.

Returns

Polyline

A Polyline representing the encoded polyline string for the provided coordinates.

Exceptions

ArgumentNullException

Thrown if `encoder` is null.